Republic of the Philippines
CAVITE STATE UNIVERSITY
Don Severino de las Alas Campus
Indang, Cavite

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY
Department of Information and Technology

COSMIC TETRIS

Web System and Technologies 2
Submitted to Sir Aimiel Peña
College of Engineering and Information Technology
Cavite State University
Indang, Cavite

Finals Requirement
for the course Web System and Technologies 2
Bachelor of Science in Information Technology

\

JOHN CARLO A. CATIBOG
JAY B. MANLIGUEZ
AHRON CHRISTIAN A. RODRIGUEZ
June 2026

TABLE OF CONTENTS

1. INTRODUCTION

This documentation covers the design, development, and technical architecture of Cosmic Tetris — a fully functional web-based implementation of the classic Tetris arcade game. The project was developed without the use of a PHP framework, relying instead on Native PHP for server-side logic, JavaScript for client-side game mechanics, HTML5 Canvas for rendering, Bootstrap 5 for responsive styling, and a cloud-hosted PostgreSQL database (Supabase) for data persistence. The sections below detail the project's objectives, design evolution, technical architecture, core logic, and the challenges encountered throughout development.

1.1 Project Overview

Cosmic Tetris is a web-based implementation of the classic Tetris puzzle game, rebuilt as a multi-user platform with persistent high-score tracking and a global leaderboard. The application is hosted on a local Apache server (XAMPP) and connects to a Supabase PostgreSQL cloud database for user and score management. The tech stack was deliberately chosen to remain framework-free on the server side, showcasing core Native PHP skills: object-oriented class design, PDO-based database operations, PHP session management for authentication, and server-side input validation.

The front end combines HTML5 Canvas — for real-time Tetris board rendering — with Bootstrap 5 for layout and a custom CSS glassmorphism theme that gives the application an immersive cosmic/space aesthetic. The game engine is written entirely in vanilla JavaScript, using requestAnimationFrame for a smooth 60 fps game loop. Score data is submitted to the server asynchronously via the Fetch API (AJAX), allowing the game to save a player's personal best without a page reload.

The project is organized into three logical layers:

- Database layer — a single PostgreSQL table (TetrisGame) hosted on Supabase, storing user credentials and scores.
- Back-end layer — a PHP class (tetrisgame) that encapsulates all database operations: registration, authentication, score updates, and leaderboard queries.

- Front-end layer — PHP view files (login, register, dashboard, game, leaderboard) and a JavaScript game engine, styled with custom CSS and Bootstrap.

1.2 Objectives and Rules

1.2.1 Objectives

The project was developed with the following SMART objectives in mind. Each objective maps directly to a feature of the final system:

- To implement a fully functional, browser-playable Tetris game using HTML5 Canvas and vanilla JavaScript, rendering a 10-column by 15-row grid with seven standard tetromino shapes.
- To build a secure user registration and authentication system using Native PHP, enforcing unique email and username constraints, and storing passwords as bcrypt hashes via PHP's password_hash() function.
- To implement persistent high-score tracking using PDO-prepared statements against a Supabase PostgreSQL database, updating a player's record only when a new score exceeds the stored personal best.
- To develop a global leaderboard that displays the top 10 players ranked by score in descending order, retrievable at any time by any authenticated user.
- To enforce session-based access control so that only authenticated users can access the game board, dashboard, and leaderboard, with automatic redirection to the login page otherwise.
- To deliver a responsive, visually engaging user interface using Bootstrap 5 and a custom cosmic/space CSS theme (glassmorphism cards, animated star background, neon-glow buttons) that works across desktop and mobile screen sizes.

1.2.2 Game Flow and Rules

Cosmic Tetris follows the standard rules of the original Tetris game with a few web-specific additions for session persistence and score saving. The complete flow from start to finish is as follows:

Game Flow:

- **Step 1 – Registration:** A new player visits register.php, enters a unique username, email, and password (confirmed twice). The system validates all fields and redirects to login on success.
- **Step 2 – Login:** The player logs in with their email and password. The server verifies credentials using password_verify() and, on success, creates a PHP session storing user_id, username, is_logged_in, and score.
- **Step 3 – Mission Control (Dashboard):** The authenticated player arrives at dashboard.php, which displays their personal best score and provides buttons to play the game or view the leaderboard.
- **Step 4 – Gameplay:** The player enters game.php where the JavaScript engine initializes. A random tetromino spawns at position (3, 0) at the top center of the 10×15 grid. The game loop begins at 400 ms per tick.
- **Step 5 – Piece Movement:** The player uses the keyboard arrow keys to move pieces left/right, accelerate their fall (Down), or rotate them 90° clockwise (Up). Movement is blocked at walls and existing blocks.
- **Step 6 – Row Clearing:** When a complete horizontal row is detected, it is removed from the board, all rows above shift down, and 100 points are added to the score per cleared row.
- **Step 7 – Speed Increase:** Every 5 seconds of gameplay, the fall interval decreases by 20 ms (minimum 500 ms), progressively increasing difficulty.
- **Step 8 – Game Over:** The game ends when a newly spawned tetromino immediately collides with existing blocks. The final score is automatically submitted to the server via a Fetch API POST request. The browser alerts the player with their final score.

- **Step 9 – Score Saving:** The server's UpdateScore() method compares the submitted score with the stored personal best. Only if the new score is strictly greater is the database record updated.
- **Step 10 – Leaderboard:** The player can view the global leaderboard (top 10) at any time. Their own rank, score, and username are displayed in a personal summary panel below the table.

Rules Summary:

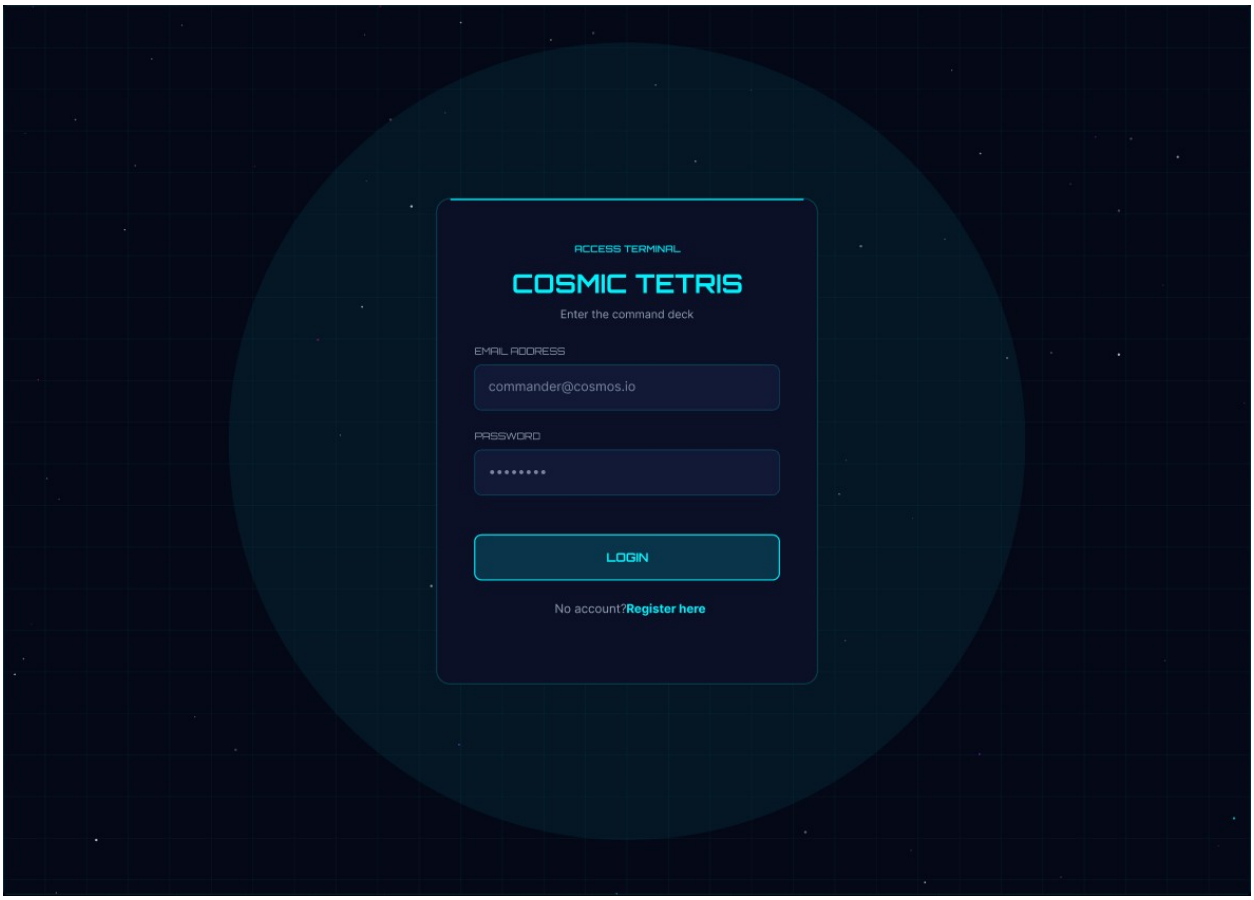
- The game grid is 10 columns wide and 15 rows tall. Each cell is 50×50 pixels.
- There are 7 standard tetromino types: I (cyan), O (blue), J (orange), L (yellow), S (green), Z (red), and T (purple).
- A "Next Piece" preview is shown to the player at all times.
- Clearing one row scores 100 points; multiple simultaneous clears multiply accordingly (e.g., 2 rows = 200 pts).
- Pieces cannot move through walls, the floor, or already-placed blocks.
- Rotation is reverted automatically if the rotated shape would cause a collision.
- The game cannot be paused in the current version. Players may press "Restart" to start a new game (score is saved first) or "Quit" to return to the dashboard.
- Only logged-in users may access the game. Unauthenticated access attempts redirect to the login page.

2. DESIGN AND PROTOTYPING

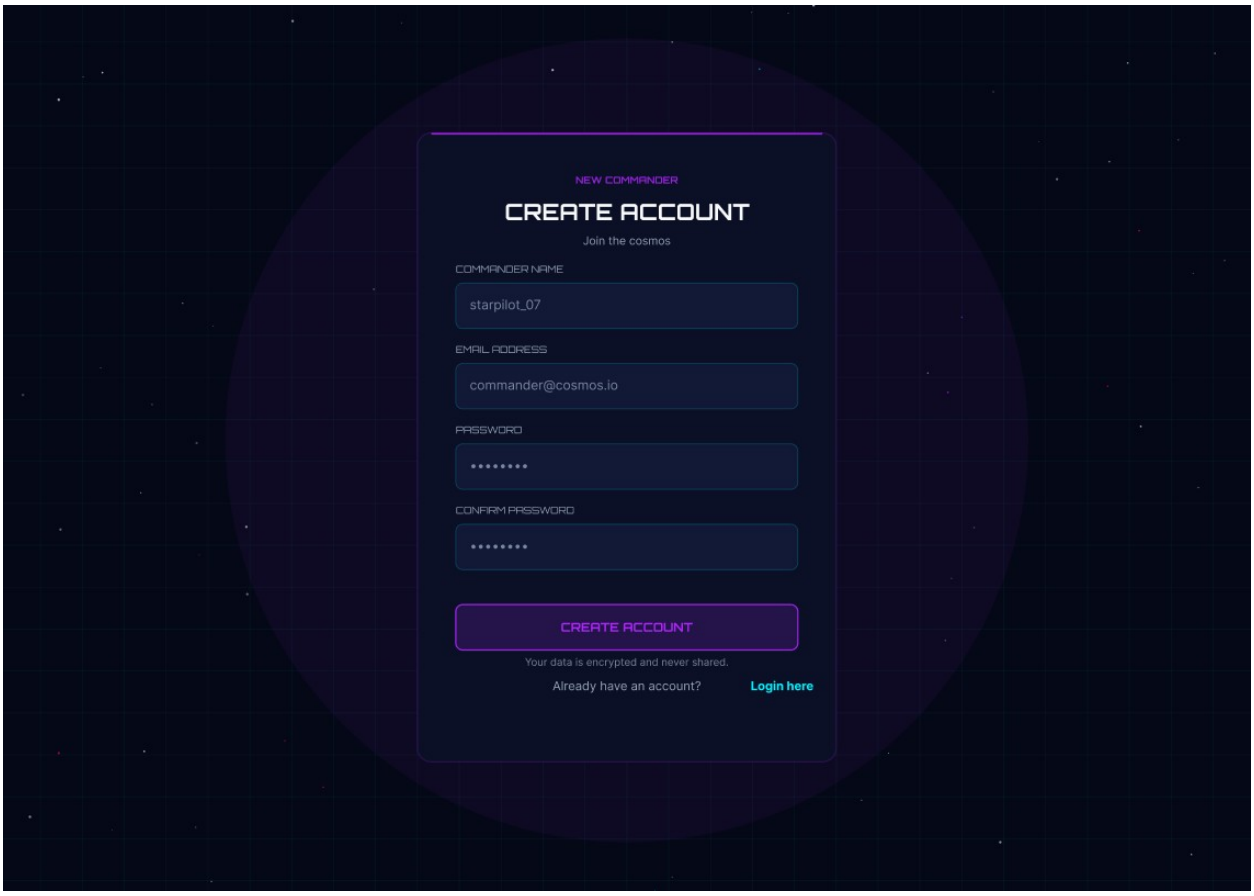
The design phase of Cosmic Tetris went through several stages, beginning with rough wireframes of the required screens and culminating in a polished glassmorphism-themed interface. The subsections below document the initial wireframe concepts, the final UI screenshots, and the most significant design change made during development.

2.1 Initial Wireframe

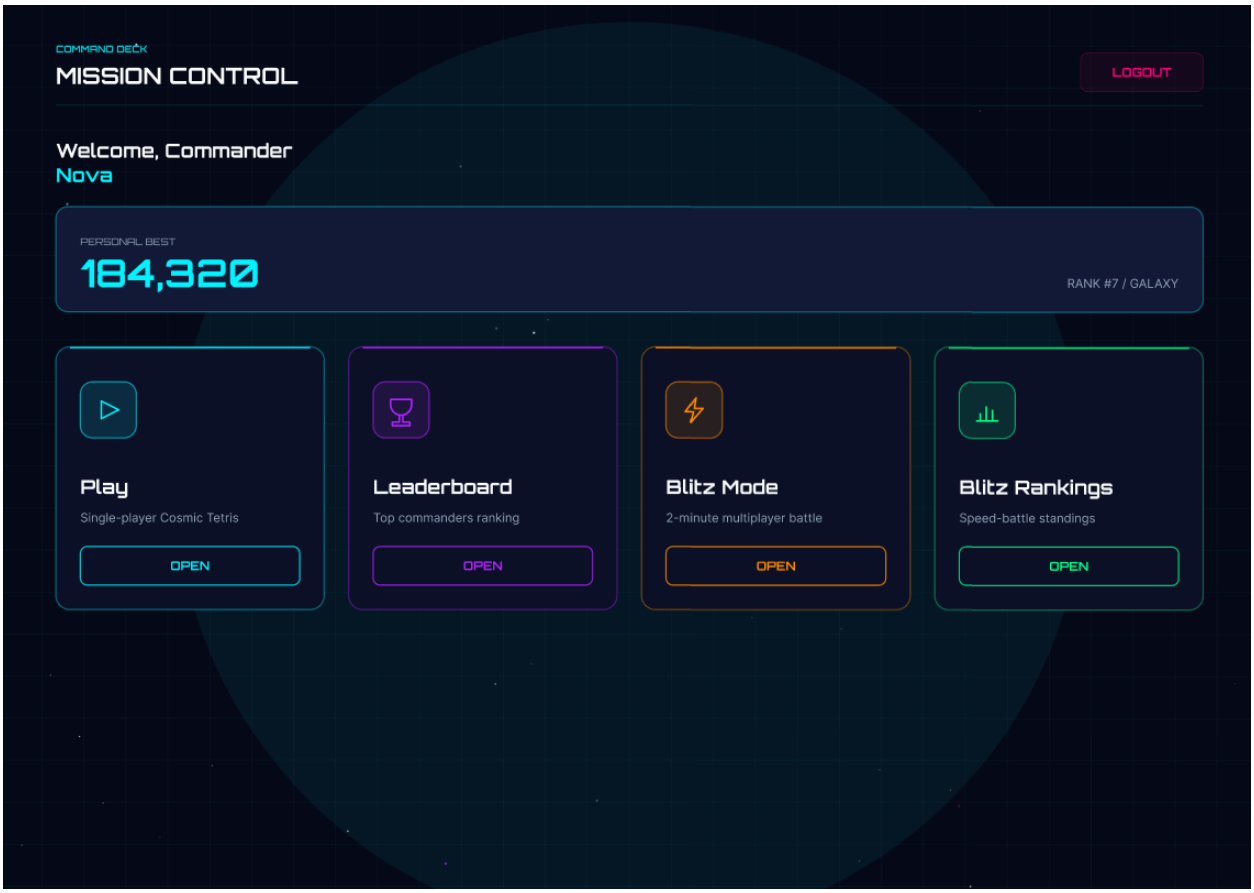
The initial wireframes captured the Figma sketches of every screen the players and administrators would interact with. Each sketch focused on layout and primary interactions rather than visual styling, and served as the blueprint for the final cosmic-themed UI. The following screens were wireframed before development began:



- **Login Screen:** Sketch of a centered card with two input fields (Email and Password), a primary Login button, and a link to the registration page for new commanders.

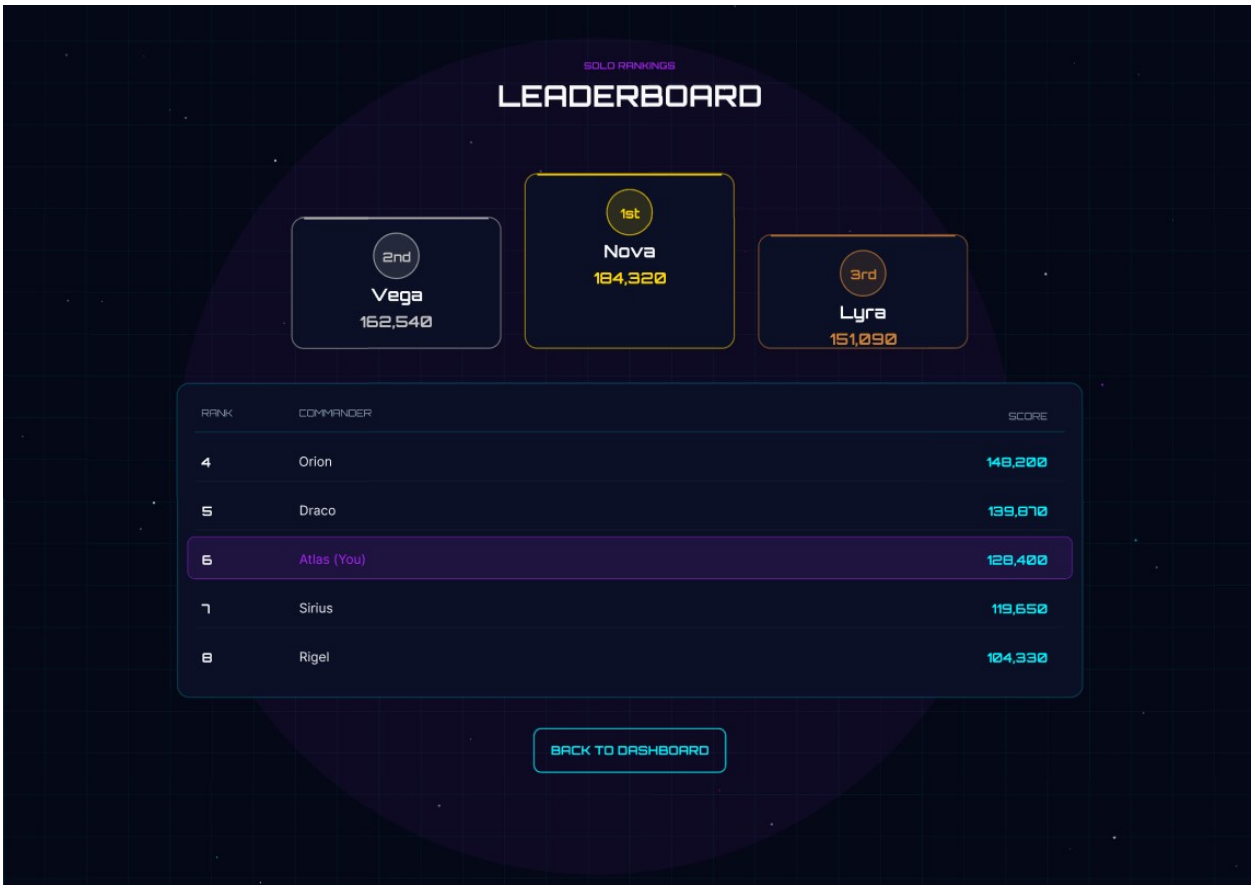


- **Register Screen:** Wireframe of a registration card with four inputs (Username, Email, Password, and Confirm Password) and a Register button that returns the user to the login screen on success.

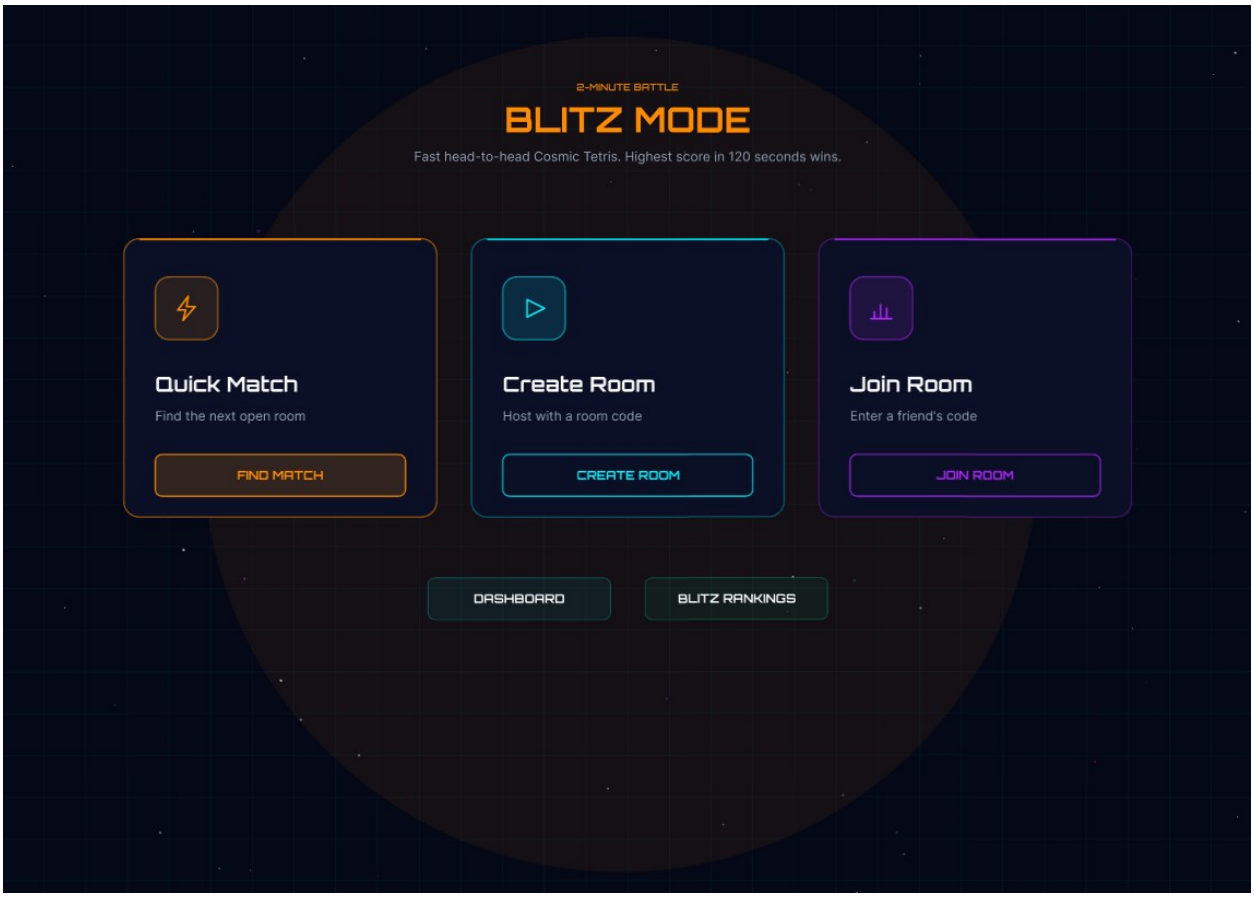


- **Mission Control Dashboard:** Sketch of the post-login hub showing the player's name, their current personal best score, and primary action buttons for Play, Leaderboard, and Logout.

- **Single Player Cosmic Tetris Game Screen:** Wireframe of the gameplay layout with the playfield in the center, the live score on the left, the next-piece preview on the right, and Restart / Quit controls below the board.

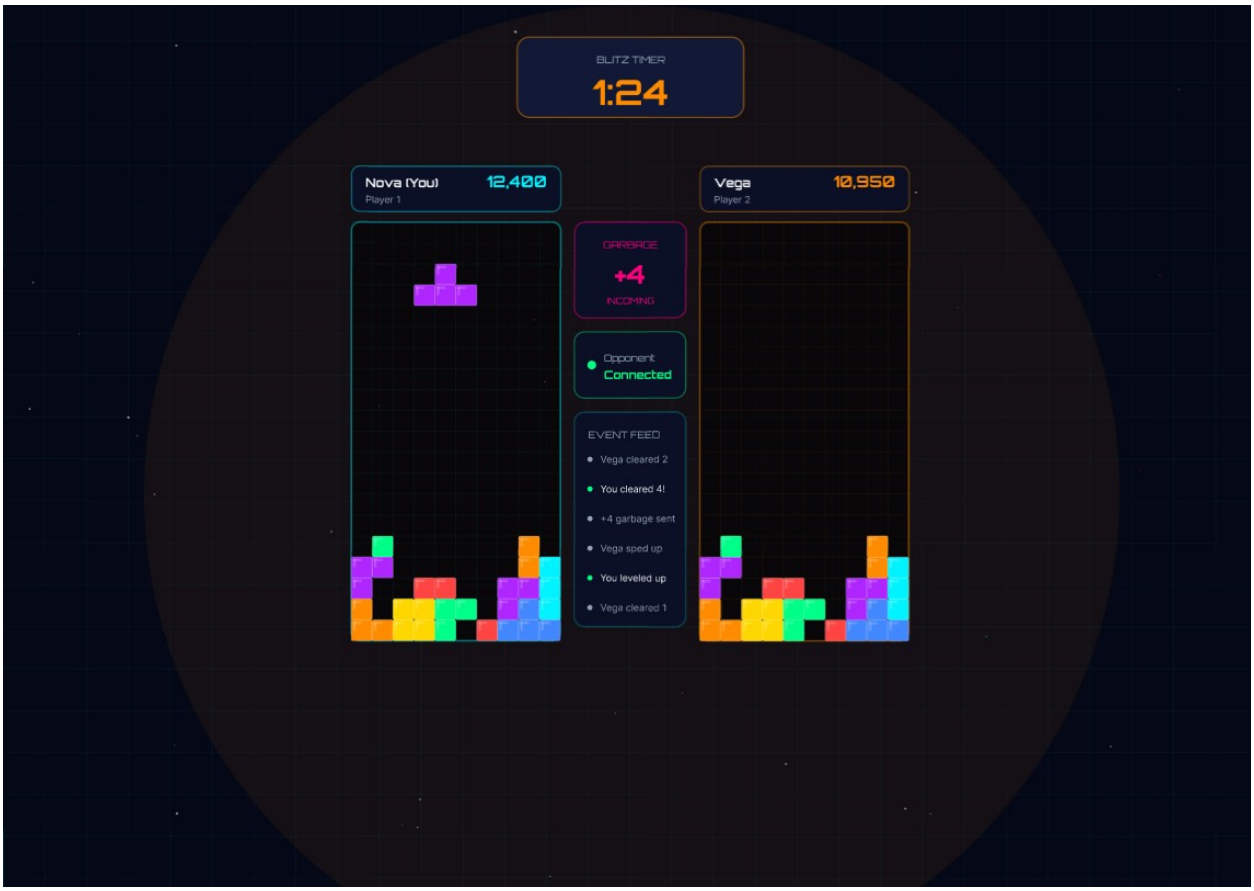


- **Leaderboard Screen:** Sketch of a top-10 ranking table with columns for Rank, Username, and Score, followed by a personal-ranking summary card showing the logged-in player's global position.



- **Blitz Mode Selection Screen:** Wireframe of the multiplayer hub presenting three entry options - Quick Match (auto matchmaking), Create Room (generates a room code to share), and Join Room (enter an existing code).

- **Blitz Room / Ready Lobby Screen:** Sketch of the pre-match waiting area showing both player names in a "vs" layout, a Ready button for each side, and a countdown indicator before the match begins.

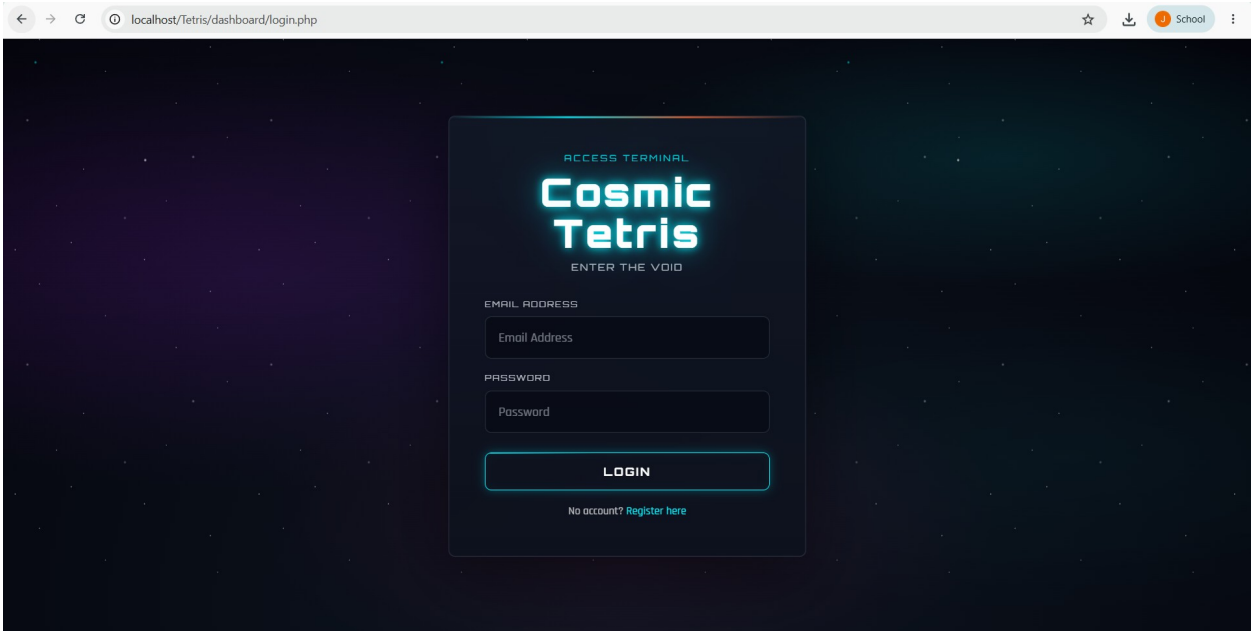


- **Multiplayer Battle Screen:** Wireframe of the side-by-side gameplay layout - the local player's board on one side and the opponent's mirrored board on the other, with a shared 2-minute timer and both players' live scores anchored at the top.

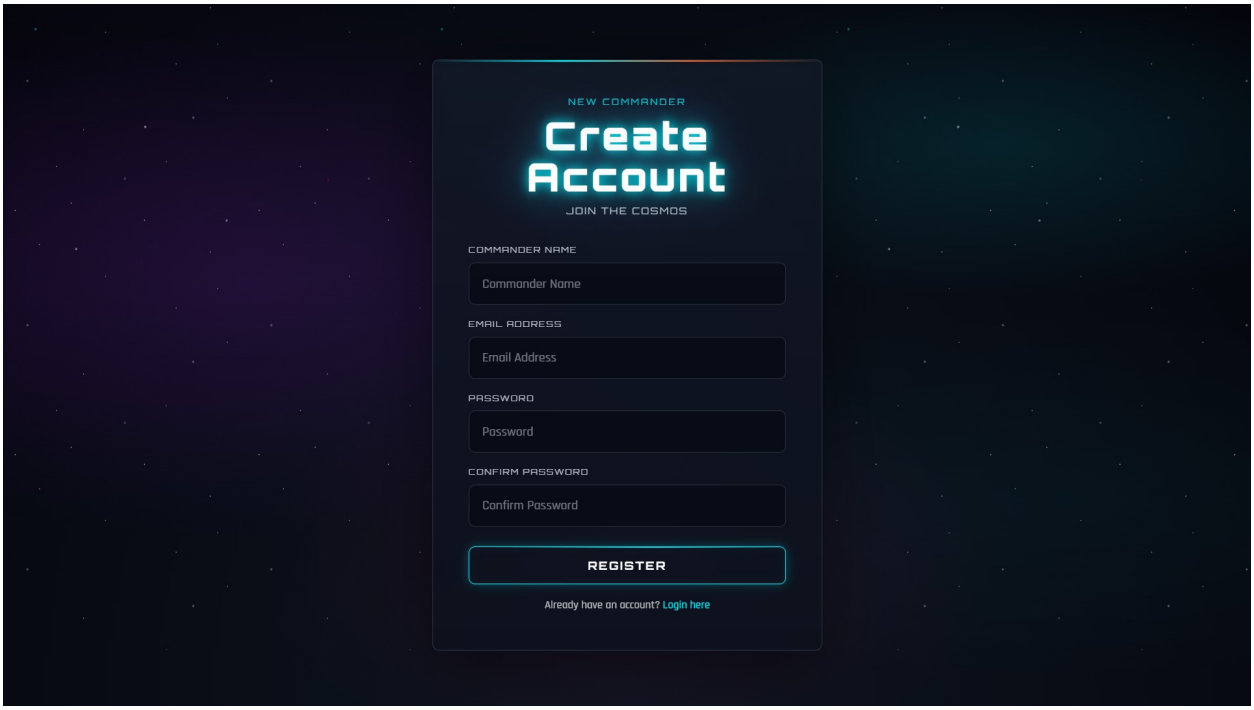
- **Game Over Modal State:** Sketch of the end-of-game popup showing the final score, the winner declaration (in Blitz Mode), and Restart / Quit (or Rematch, in Blitz Mode) buttons.

2.2 Final UI Design

The final UI adopted a cosmic/space theme with glassmorphism-style cards, an animated star-field background, and neon-glow interactive elements. Google Fonts "Orbitron" (titles and buttons) and "Rajdhani" (body text) were used to reinforce the sci-fi aesthetic. The color palette is anchored by three neon accents: Cyan (#00F3FF), Purple (#B026FF), and Pink (#FF007F), set against a near-black deep-space gradient background.

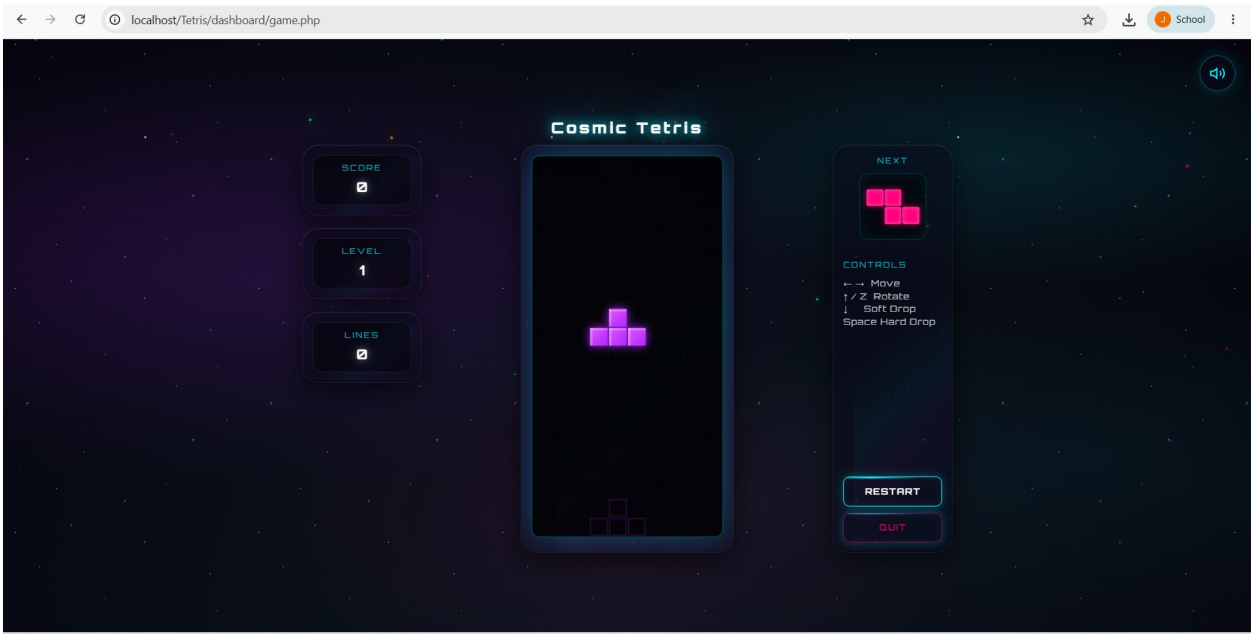


- **Login Page (login.php):** A centered glass card on a starfield background. Fields for Email and Password, a cyan neon "Login" button, and a link to the registration page. Error messages appear below the form in red.

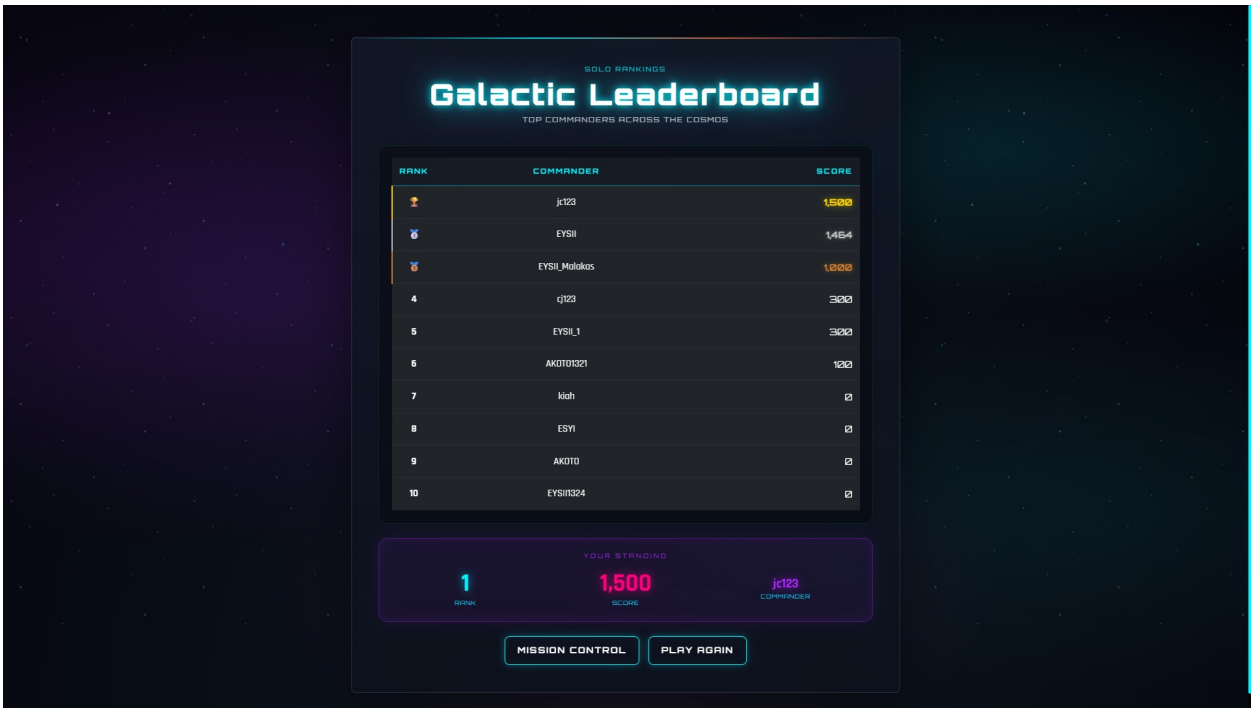


- **Register Page (register.php):** Identical layout to login with four fields: Username, Email, Password, and Confirm Password. Validation errors are shown inline.

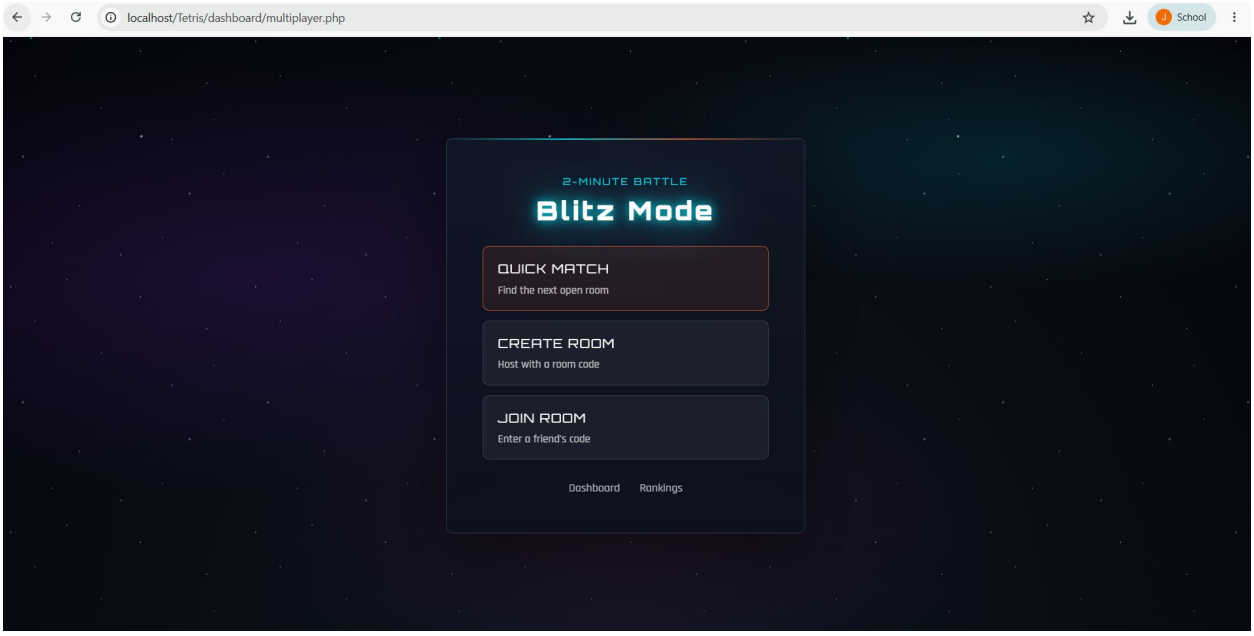
- **Dashboard / Mission Control (dashboard.php):** A glass card with the "Mission Control" header, a glowing cyan box displaying the player's personal best score, and two neon buttons for Play and Leaderboard. A text logout link sits at the bottom.



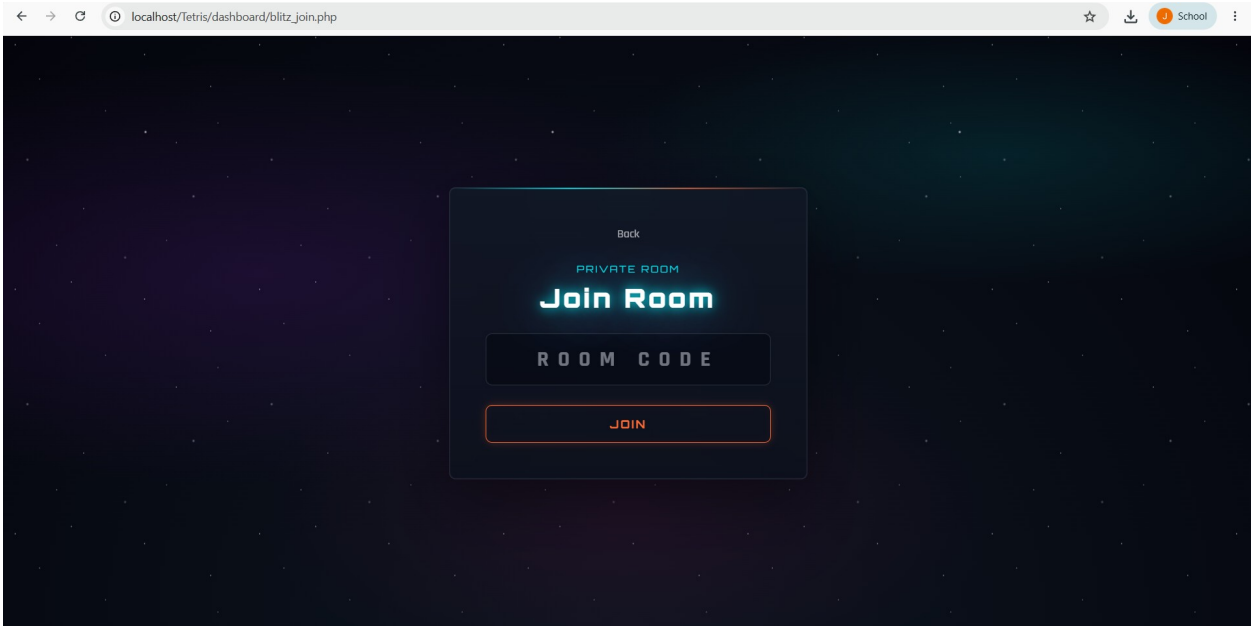
- **Game Page (game.php): A** three-panel responsive layout. The left panel shows the live score; the center panel contains the Tetris canvas with a grid overlay; the right panel shows the next piece preview and Restart/Quit buttons.



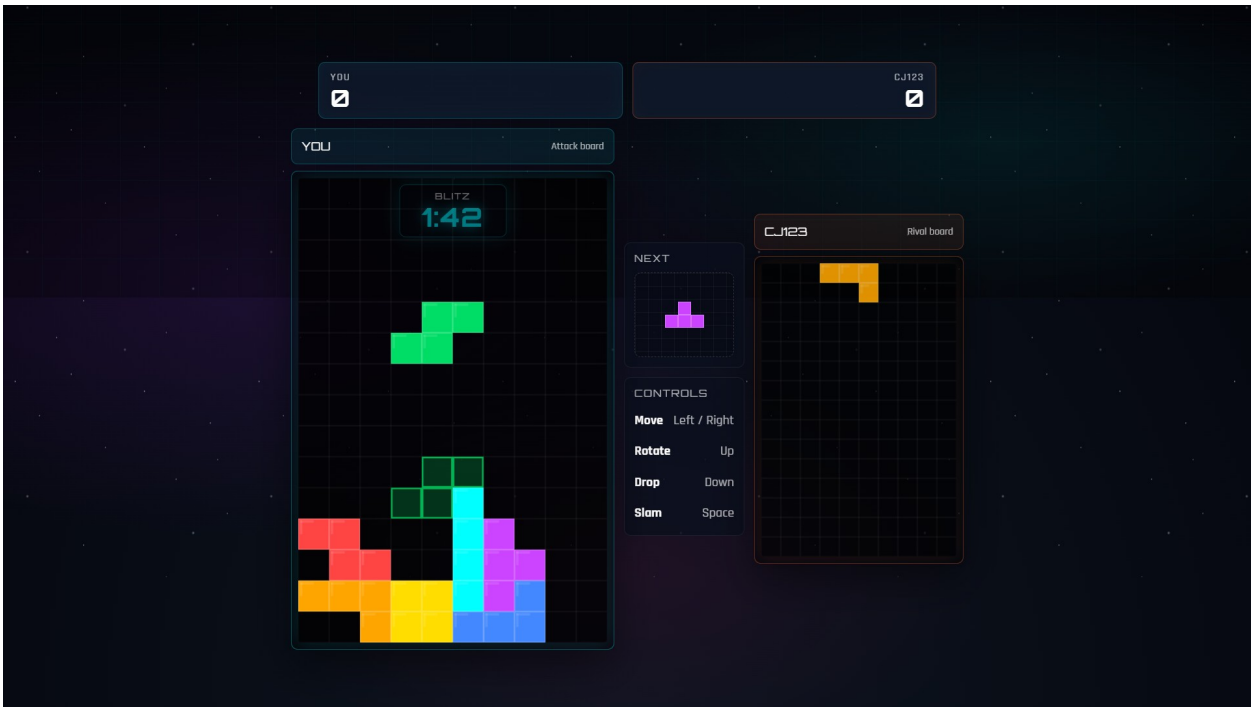
- **Leaderboard (leaderboard.php):** A dark-themed striped table with cyan column headers listing the top 10 commanders by score, followed by a purple-bordered personal rank card showing the logged-in player's rank, score, and username.



- **Blitz Mode Menu (multiplayer.php):** A hub for multiplayer flows. Offers Quick Match (auto-matchmaking), Create Room (generates a 6-character room code), and Join Room (enter an existing code). Uses orange neon-blitz buttons to visually distinguish multiplayer entry points from the cyan solo flow.



- **Blitz Join Room (blitz_join.php):** A focused screen with a single input accepting a 6-character room code. Submits to the Blitz API and displays inline errors (room not found, room full, match already in progress) without a page reload.

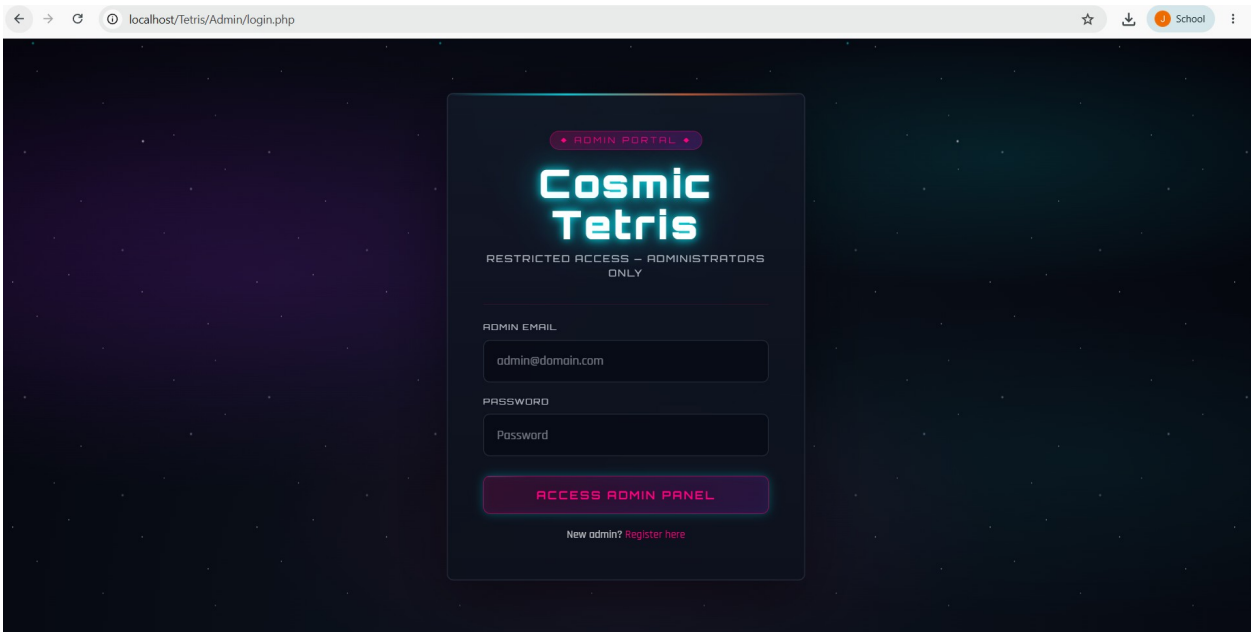


- **Blitz Room (blitz_room.php):** The live multiplayer arena, driven by a phase machine - Waiting (waiting for an opponent to join), Ready (both players must confirm), Countdown (animated 3-2-1 overlay), Game (side-by-side 10x15 canvases with the player's own board on the left and the opponent mirror on the right, a synchronized blitz timer, garbage-line attacks on line clears, and a center HUD), Result (winner declared with a rematch option), Cooldown (brief transition between rounds), and Error (a recoverable error overlay with a return-to-menu fallback).

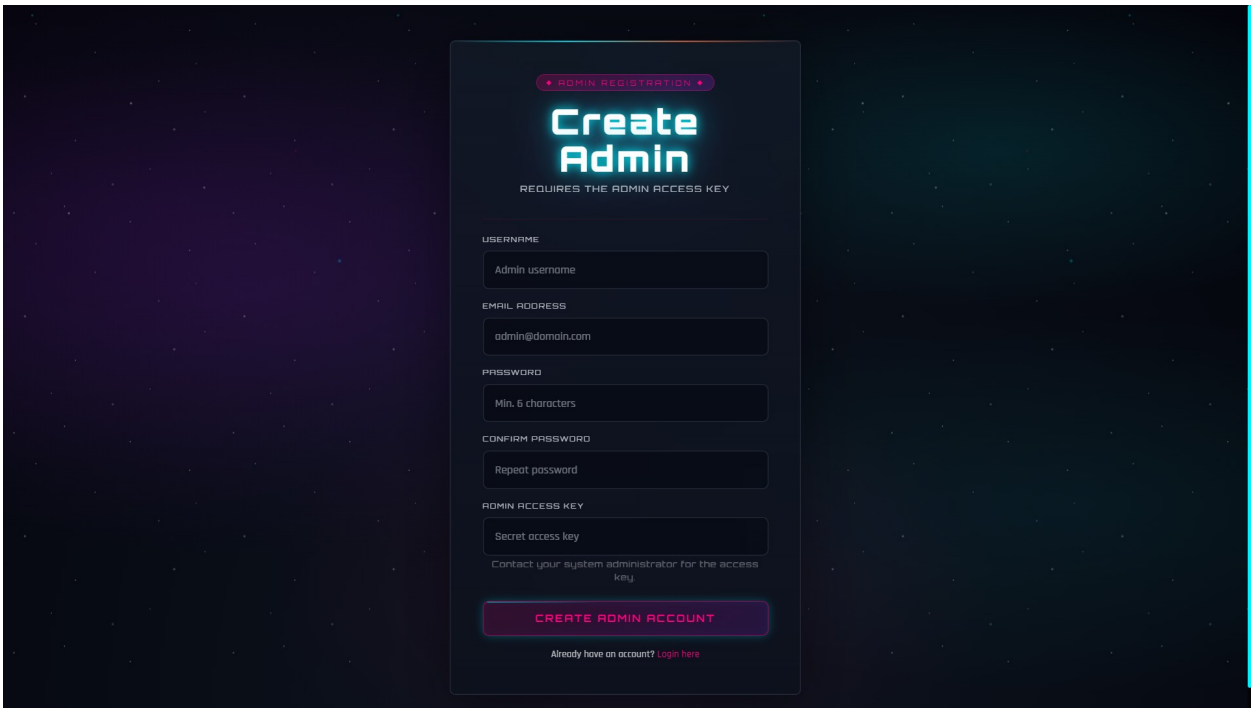
- **Blitz Leaderboard / Rankings (blitz_leaderboard.php):** A standalone ranking table that mirrors the solo leaderboard layout but ranks players by Blitz wins. It uses the orange neon-blitz palette and an orange-bordered personal rank card to clearly separate competitive Blitz standings from the cyan solo board.

2.3 Admin Side

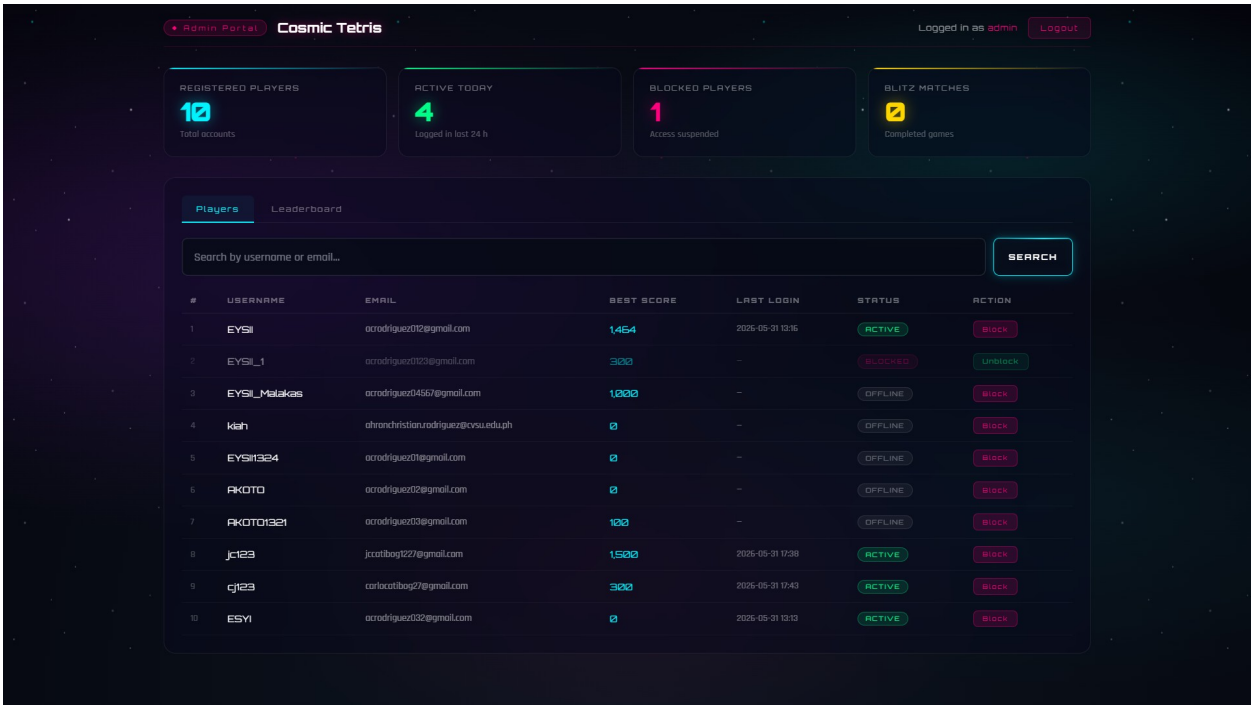
The Cosmic Tetris administrative interface is a separate area under the /Admin/ directory, intended for operators rather than players. It is gated by its own session bootstrap (backEnd/admin_session.php) and a dedicated admin_users table; an authenticated player session does not grant admin access. The admin UI reuses the cosmic theme but leans on the pink/magenta accent (#FF007F) instead of the cyan player palette, so an operator can never confuse the two surfaces.



- **Admin Login (Admin/login.php):** A glass card login form scoped to administrators. Credentials are validated against the admin_users table via PDO-prepared statements and password_verify(); on success the session is regenerated and admin_logged_in is set in $_SESSION. Failed attempts produce an inline error.



- **Admin Register (Admin/register.php):** A bootstrap-only form for creating an administrator account, typically used during initial setup. Enforces unique username/email and stores the password as a bcrypt hash via password_hash().



- **Admin Dashboard (Admin/dashboard.php):** The control panel. A top stats strip surfaces total players, currently-active players, blocked players, and total Blitz games played. Below it a searchable, paginated player roster lists every registered player with a row-level Block / Unblock action (both CSRF-protected with a per-session admin_csrf token). Two side-by-side panels render the Solo top-10 and Blitz top-10 leaderboards so administrators can audit scores in one view.
- **Admin Logout (Admin/logout.php):** Unsets the admin session, expires the session cookie, calls session_destroy(), and redirects back to Admin/login.php.

  Player-management actions taken from the admin dashboard (block / unblock) immediately affect the next login attempt by that player: a blocked account is rejected at the tetrisgame::LoginUser() step with a generic "Account unavailable" message, which also prevents the block from being used as a user-enumeration oracle.

2.4 Design Pivot

The most significant design change between the initial wireframe and the final product was the shift from a plain, functional HTML layout to a full immersive cosmic theme.

Original Plan: The wireframes envisioned a simple, Bootstrap-default layout with standard form cards, white background, and minimal styling. The focus was entirely on functionality: get the game working, save scores, show a leaderboard. Visual design was treated as secondary.

What Changed: During development, the team decided that a Tetris game — by its nature an engaging, arcade-style experience — deserved a UI that matched its energy. The team introduced a deep-space radial gradient background (#1B2735 to #090A0F), a CSS keyframe animation that scrolls a repeating star pattern upward infinitely, glassmorphism cards (backdrop-filter: blur(15px) with semi-transparent backgrounds), and neon-glow buttons with box-shadow animations on hover. Custom CSS variables (--neon-blue, --neon-purple, --neon-pink) were defined in :root to keep the color system consistent and easy to modify.

Justification: This pivot significantly improved the perceived quality and cohesiveness of the product. The "cosmic commander" framing (users are called "Commander", the dashboard is "Mission Control", the leaderboard is the "Galactic Leaderboard") created a narrative layer that made the game more engaging without adding any technical complexity to the back-end logic.

3. TECHNICAL ARCHITECTURE

This section documents how the application's components are organized, how HTTP requests travel through the system, the database structure, and how PHP sessions maintain state across page loads.

3.1 System Life Cycle

Cosmic Tetris uses a straightforward request-response cycle with Native PHP. There is no MVC router; each PHP file handles both the logic for its own form submissions (via $_POST) and the rendering of its HTML view. The cycle for a typical authenticated request is as follows:

Full Request Flow (Login Example):

- 1. Browser sends GET request to login.php. PHP calls session_start(), outputs the HTML login form.
- 2. User submits the form. Browser sends POST request to login.php with email and password in the request body.
- 3. login.php checks $_POST['login']. It validates that both fields are non-empty, then instantiates the tetrisgame class.
- 4. tetrisgame::LoginUser() uses dbconnect::dbconnect() to open a PDO connection to the Supabase PostgreSQL database over port 6543 (pgBouncer connection pooler).
- 5. A prepared statement (SELECT * FROM "TetrisGame" WHERE email = ?) is executed. If the row exists, password_verify() checks the submitted password against the stored bcrypt hash.
- 6. On success: $_SESSION['user_id'], $_SESSION['username'], $_SESSION['is_logged_in'], and $_SESSION['score'] are set. PHP issues header("Location: dashboard.php") and exits.
- 7. Browser follows the redirect to dashboard.php, which calls session_start() and checks $_SESSION['is_logged_in']. If true, the dashboard HTML is rendered with the player's personal best score.

Score-Saving AJAX Flow (Game Over):

- 1. When the game ends (or the player quits), script.js calls saveHighScore(), which issues a Fetch API POST to dashboard.php with JSON body { score: <value> }.

- 2. dashboard.php reads the raw input with file_get_contents("php://input") and decodes the JSON. If the score is greater than 0, it calls tetrisgame::UpdateScore().
- 3. UpdateScore() queries the current stored score for the user, and only issues an SQL UPDATE if the new score is strictly higher.
- 4. dashboard.php returns a JSON response { success: true, score: <updatedScore> }. JavaScript logs the result and then redirects the browser to dashboard.php for a normal page load.

3.2 Database Schema

The application uses a single PostgreSQL table named TetrisGame, hosted on a Supabase cloud database. The table is deliberately denormalized to a single entity: the user account, which also stores the player's personal best score. This design is appropriate for the scope of the project since scores are not historical — only the current personal best is retained.

Table: TetrisGame

Column	Data Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier for each user account.
email	VARCHAR	UNIQUE, NOT NULL	User's email address. Checked for uniqueness before every INSERT.
username	VARCHAR	UNIQUE, NOT NULL	Player's display name. Shown on the leaderboard and dashboard.
password	VARCHAR	NOT NULL	bcrypt hash of the user's password, generated by PHP's password_hash(PASSWORD_DEFAULT).

Column	Data Type	Constraints	Description
score	INTEGER	NULLABLE, default 0	Personal best score. NULL on registration; updated only when a new score exceeds the stored value.

Table 1: TetrisGame PostgreSQL schema (Supabase cloud database)

SQL CREATE TABLE equivalent:

```
CREATE TABLE "TetrisGame" (
    id       SERIAL       PRIMARY KEY,
    email    VARCHAR(255)  UNIQUE NOT NULL,
    username VARCHAR(100)  UNIQUE NOT NULL,
    password VARCHAR(255)  NOT NULL,
    score    INTEGER       DEFAULT 0
);
```

The connection is established in dbConnect/dbconnect.php using PHP's PDO (PHP Data Objects) library with the pgsql driver. The host points to Supabase's AWS ap-southeast-2 connection pooler (port 6543) rather than the direct database port (5432), which is required when using Supabase's pgBouncer transaction pooler for stateless PHP connections. PDO::ATTR_ERRMODE is set to PDO::ERRMODE_EXCEPTION so that all database errors throw catchable PDOExceptions rather than silently failing.

3.3 State Management

PHP sessions ($_SESSION) are the sole mechanism for maintaining state across page requests. Since HTTP is stateless, the session keeps track of who is logged in, their display name, and their current best score without requiring a database query on every

page load. Every protected page calls session_start() at the very top before any HTML is output.

Session Variables Used:

Session Key	Type	Purpose
$_SESSION['user_id']	Integer	Stores the database primary key of the authenticated user. Set on login; used to identify the user server-side.
$_SESSION['username']	String	Player's display name. Used to query scores, display on dashboard, and pass to leaderboard ranking functions.
$_SESSION['is_logged_in']	Boolean (true)	The authentication gate. Every protected page checks this; if absent or false, the user is redirected to login.php.
$_SESSION['score']	Integer	Cached personal best score for the current session. Updated when UpdateScore() stores a new high score to avoid an extra DB query on the dashboard.
$_SESSION['msg']	String	Flash message passed between redirects (e.g., "Email already exists.", "Registration successful."). Displayed once and then unset().

Table 2: PHP $_SESSION variables used across the application

How session security is maintained:

- session_start() is called at the top of every PHP file that reads or writes session data, before any output.
- The authentication check on dashboard.php and leaderboard.php is: if (! isset($_SESSION['is_logged_in']) || $_SESSION['is_logged_in'] !== true). This double-checks both existence and value, preventing a tampered session variable from granting access.
- On logout, both session_unset() and session_destroy() are called to ensure all session data is cleared from the server and the session cookie is invalidated.
- Flash messages ($_SESSION['msg']) are immediately unset() after being displayed, so they cannot be replayed on a subsequent page refresh.
- The score AJAX endpoint (dashboard.php) reads $_SESSION['username'] to identify the user. If the session has expired or is missing, $username becomes an empty string, and the UpdateScore() query silently returns the existing score without making any changes.

3.4 Match Lifecycle and Room Phases

A Blitz match progresses through seven discrete UI phases. The blitz_room.php page is a single document that contains all phase templates (phase_waiting, phase_ready, phase_countdown, phase_game, phase_result, phase_cooldown, phase_error). JavaScript toggles visibility with the showPhase() helper so that only one phase is displayed at a time. The flow is identical regardless of whether the match was started via Quick Match, Create Room, or Join Room.

Phase sequence for a typical match:

Phase 1 – Waiting: After entering blitz_room.php, the client calls the blitz_api with action=create, action=find, or action=join. If no opponent is yet present, the room code is displayed and the lobby is polled every 1.2 seconds until a second player joins.

Phase 2 – Ready: Once both players are in the room, both see a "VS" matchup card with each commander's name. A 20-second countdown bar ensures inactive players cannot stall the room indefinitely. Each player presses Ready! which sets the p1_ready or p2_ready flag in the database.

Phase 3 – Countdown: When both flags equal 1, the server transitions status to 'playing' and a synchronized 5-second countdown begins on both clients ("5, 4, 3, 2, 1, GO!").

Phase 4 – Game: Two canvases are rendered side-by-side. The player's own board uses a 38 px block size; the opponent's mirror board uses 24 px. The 2:00 timer ticks down centrally, and the syncRoom() call uploads the player's board snapshot and downloads the opponent's at 100 ms intervals (10 Hz).

Phase 5 – Result: When either the timer hits zero or both players have topped out, the room status flips to 'finished' and the winner is computed server-side. Both clients show the result screen with both scores and the rematch controls.

Phase 6 – Cooldown: Reserved for matchmaking back-off scenarios (for example, a player who hit Find Match but had no rooms available).

Phase 7 – Error: A friendly error screen with a Retry button is shown for any unrecoverable setup error (database unavailable, missing blitz tables, network timeout, expired ready timer).

3.5 Matchmaking Modes

The multiplayer.php landing page exposes three entry points into blitz_room.php, each driving a different value of the mode query parameter. The mode is validated server-side in bootstrap.php and constrained to one of the three values below.

Quick Match (mode=quick): blitz_api->quickFind() searches for an open room created by another user within the last 5 minutes whose host has pinged in the last 60 seconds. If a match is found the caller joins it; otherwise a brand-new room is created and the caller waits to be matched.

Create Room (mode=create): Allocates a new private room and displays its 6-character code. The code is drawn from a 24-letter alphabet (no ambiguous I or O), yielding roughly 191 million unique combinations.

Join Room (mode=join): The blitz_join.php page presents an input field that auto-normalizes the entered code to uppercase letters. On submit it redirects to blitz_room.php?mode=join&code=XXXXXX. The bootstrap script rejects codes whose length is not exactly 6.

3.6 Real-Time Synchronization Loop

Because the project intentionally avoids WebSockets and any non-PHP back-end, the live state of both players is exchanged through short-interval HTTP polling. Every 100 ms (10 Hz) each client issues a POST to backEnd/blitz_api.php?action=sync with its current board snapshot, score, alive status, and any new garbage to send. The same response carries the opponent's board, score, alive flag, accumulated garbage, room status, winner, and a disconnect indicator.

Key implementation details that make 10 Hz polling viable in a Native PHP + Supabase environment:

session_write_close() is called immediately after the auth check in blitz_api.php. PHP's default file-based session handler holds an exclusive flock for the lifetime of the request, which would serialize every request from the same browser. Releasing it early raises the real concurrency ceiling for the sync loop.

A syncInFlight boolean guards against overlapping requests. If the previous sync has not completed when the 100 ms tick fires, the next tick is skipped instead of pile-driving the server.

Each fetch is bounded by a 4-second AbortController timeout, so a stalled request cannot leave syncInFlight stuck at true and freeze the loop forever.

After a tetromino locks, syncRoom() is called immediately instead of waiting for the next 100 ms tick — the lock is the biggest change the opponent will see, so pushing it eagerly cuts perceived latency.

The opponent's board snapshot received from the server is cached in oppBoardCache and re-rendered every animation frame (60 Hz). This makes the rival canvas paint smoothly at the display refresh rate rather than stepping at the 10 Hz sync rate.

The boardSnapshot() function overlays the active falling piece on top of the locked board before uploading, so the opponent sees the live trajectory of every piece in flight, not just the settled blocks.

Server-side, blitz_api.php wraps every connection attempt in a 2-pass retry loop that detects transient PostgreSQL errors ("server closed the connection unexpectedly", "terminating connection", SSL drops) and reopens the pool before the request fails.

Roughly one in every ten writes triggers a stale-room cleanup that deletes any blitz_rooms record older than 15 minutes, keeping the table from growing without bound across abandoned sessions.

3.7 Blitz Database Schema

Blitz adds two new tables to the Supabase database. The SQL is shipped in blitz_tables.sql at the project root and is meant to be pasted once into the Supabase SQL Editor when the application is first deployed.

Table: blitz_rooms (live match state)

Column	Type	Constraints	Description
room_code	VARCHAR(8)	PRIMARY KEY	Unique 6-letter

			room identifier.
p1_username	VARCHAR(100)	NOT NULL	Host commander's display name.
p1_board	TEXT	DEFAULT '[]'	JSON-encoded 10x15 board snapshot for player 1.
p1_score	INTEGER	DEFAULT 0	Player 1's current score.
p1_ready	INTEGER	DEFAULT 0	1 once player 1 presses Ready.
p1_alive	INTEGER	DEFAULT 1	0 if player 1 has topped out.
p1_garbage	INTEGER	DEFAULT 0	Total garbage lines sent by player 1.
p2_username	VARCHAR(100)	NULL until joined	Guest commander's display name.
p2_board / p2_score / p2_ready / p2_alive / p2_garbage	various	—	Mirror of the player 1 columns for player 2.
status	VARCHAR(20)	DEFAULT 'waiting'	'waiting', 'ready', 'playing', or 'finished'.
winner	VARCHAR(100)	NULL until finished	Username of the winning commander, or NULL on draw.
rematch_code	VARCHAR(8)	NULL	New room code allocated by the

			rematch flow.
rematch_count	INTEGER	DEFAULT 0	Number of rematches played in this pairing (capped at 2).
rematch_requested_by / rematch_requested_at / rematch_declined	various	—	Rematch invite metadata. See section 7.6.
created_at / p1_updated / p2_updated	TIMESTAMPTZ	DEFAULT NOW()	Heartbeats used for matchmaking freshness and disconnect detection.

Table: blitz_leaderboard (cumulative head-to-head stats)

Column	Type	Constraints	Description
username	VARCHAR(100)	PRIMARY KEY	Commander's display name (foreign-keyed by convention to TetrisGame.username).
wins	INTEGER	DEFAULT 0	Lifetime Blitz wins.
losses	INTEGER	DEFAULT 0	Lifetime Blitz losses.
best_score	INTEGER	DEFAULT 0	Highest score this commander has ever posted in a Blitz match.
total_games	INTEGER	DEFAULT 0	wins + losses + draws.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last time this row was upserted.

3.8 Blitz API Endpoint Reference

All Blitz interactions go through a single PHP entry point, backEnd/blitz_api.php, dispatched on the action query parameter. Every response is JSON, with errors surfaced as { error: "..." } and HTTP status codes where appropriate.

Action	Purpose
create	Create a new room with the caller as p1. Returns the 6-letter room code.
find	Quick-match: join the oldest open waiting room, or create a new one if none are available.
join	Join an existing room by code as p2. Validates capacity and game state.
poll	Return the room snapshot (opponent name, ready flags, status, rematch state). Updates the caller's heartbeat.
ready	Mark the caller as ready. Once both flags are 1, status flips to 'playing'.
sync	Upload the caller's board snapshot, score, alive flag, and garbage; receive the opponent's mirror snapshot. The 10 Hz heartbeat of the match.
end	Report the caller's final score and end reason (time_up, topped_out, disconnected). The server decides whether to finalize the room.
leave	Cleanly exit a room: removes the host if waiting alone, kicks the guest if pre-game, otherwise no-op.
rematch_request	Allocate a rematch room and post the

	invite to the opponent.
rematch_accept	Accept the pending invite and join the new room as p2.
rematch_decline	Decline the invite, clear pointers, and clean up the orphan rematch room.
leaderboard	Return the top 15 Blitz commanders ordered by wins, then best_score.

3.9 Blitz Leaderboard Page

The Blitz rankings live at dashboard/blitz_leaderboard.php, separate from the existing Galactic Leaderboard so single-player and competitive stats are not conflated. The page renders a top-15 table sorted by wins (descending) and then by best_score (descending). The logged-in player's own row is highlighted with a tinted background and a "You" badge. Above the table, a personal-stats box shows the player's lifetime Wins / Losses / Best Score / Games. Trophy emojis are awarded for ranks 1–3. From the dashboard a dedicated "Blitz Rankings" button takes commanders straight to this page.

4. DEVELOPMENT AND SYSTEM LOGIC

This section provides a deep dive into the core PHP and JavaScript logic powering Cosmic Tetris, covering input validation and sanitization, authorization controls, and how the code handles edge cases that could otherwise break gameplay or compromise data integrity.

4.1 Data Integrity & Validation

All user input passes through multiple validation layers before it reaches the database. Validation is applied both at the PHP level (server-side) and structurally via PDO prepared statements, which eliminate SQL injection at the driver level.

Layer 1 – Empty Field Check (register.php and login.php):

Before instantiating the tetrisgame class, each form's PHP handler verifies that no submitted field is empty:

```
if (empty($_POST['email']) || empty($_POST['username']) ||
    empty($_POST['password']) || empty($_POST['repassword'])) {
    $_SESSION['msg'] = "All fields are required.";
}
```

PHP's empty() returns true for null, empty strings, "0", and arrays with no elements, making it an effective first-pass guard. This prevents the database class from being called with incomplete data and provides an immediate, user-friendly error message.

Layer 2 – Uniqueness Checks (RegisterUser in tetrisgame.php):

Before inserting a new user, the system checks both the email and username for uniqueness with separate prepared statements:

```
$stmt = $this->db->prepare('SELECT email FROM "TetrisGame" WHERE email = ?');
$stmt->execute([$email]);
if ($stmt->rowCount() > 0) {
    $_SESSION['msg'] = "Email already exists.";
    header("Location: ../dashboard/register.php");
    exit();
}
```

The same pattern is applied for username. Using separate queries (rather than a combined OR) gives specific, actionable error messages rather than a generic "duplicate entry" failure.

Layer 3 – Password Confirmation and Hashing:

Passwords are compared with strict equality (===) before hashing to prevent type-juggling exploits:

```
if ($password === $repassword) {
    $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
    // INSERT $hashedPassword into database
}
```

PASSWORD_DEFAULT uses bcrypt with a cost factor of 10, producing a 60-character hash that is unrecoverable without brute-force. On login, password_verify($submitted, $storedHash) handles the comparison securely, including timing-safe comparison to resist timing attacks.

Layer 4 – Prepared Statements (All Database Operations):

Every query in tetrisgame.php uses PDO's prepare() and execute() pattern with positional (?) placeholders. PDO separates SQL code from data, making SQL injection structurally impossible regardless of what the user submits. Example from LoginUser():

```
$stmt = $this->db->prepare('SELECT * FROM "TetrisGame" WHERE email = ?');
$stmt->execute([$email]);
```

Layer 5 – Output Escaping (leaderboard.php):

All user-supplied data rendered into HTML (usernames and scores on the leaderboard) passes through htmlspecialchars(), which converts characters like <, >, &, and " into safe HTML entities, preventing Cross-Site Scripting (XSS) attacks:

```
echo "<td>" . htmlspecialchars($entry['username']) . "</td>";
echo "<td>" . htmlspecialchars($entry['score']) . "</td>";
```

Layer 6 – Score Validation (AJAX Endpoint in dashboard.php):

The AJAX score submission endpoint guards against zero and negative score submissions with a simple numeric check before calling UpdateScore():

```
$newscore = $data['score'] ?? 0;
if ($newscore > 0) {
    $updatedScore = $auth->UpdateScore($username, $newscore);
}
```

Additionally, UpdateScore() internally queries the current stored score and only issues an UPDATE if the new score is strictly greater — preventing a player from overwriting a high score with a lower one, even via a crafted API call.

4.2 The "Blind" Admin Restriction

Cosmic Tetris does not have a traditional admin panel with user management capabilities; all users are equal in terms of database-level permissions. However, the system implements a comparable separation of concerns through two architectural restrictions that prevent any user — or even a direct API caller — from performing unauthorized data modifications.

Restriction 1 – Server-Side Score Arbitration:

The game engine runs entirely in the client's browser (JavaScript). A technically savvy player could theoretically open the browser's developer tools and call the saveHighScore() function with an arbitrarily large score value. The server neutralizes this threat through the UpdateScore() method, which enforces a server-side rule: a score is only accepted if it is strictly greater than the player's current stored personal best:

```
$stmt = $this->db->prepare('SELECT score FROM "TetrisGame" WHERE username = ?');
$stmt->execute([$username]);
$currentScore = $stmt->fetchColumn() ?? 0;

if ($score > $currentScore) {
```

```
        $stmt = $this->db->prepare('UPDATE "TetrisGame" SET score = ? WHERE
    username = ?');
        $stmt->execute([$score, $username]);
        $_SESSION['score'] = $score;
        return $score;
    }
    return $currentScore; // reject the submission silently
```

This means the server retains final authority over what score is stored. The client can only ever propose a score; the server decides whether to accept it. A player cannot decrease another player's score, and cannot bypass this check because all UPDATE queries in the entire codebase are scoped to the session's own username (WHERE username = ?).

Restriction 2 – Session-Scoped Writes:

All write operations (UpdateScore) use $_SESSION['username'] as the identifier, which is set exclusively by the server during a successful login. A user cannot modify their session from the client side in any way that would let them impersonate another user, because the session data is stored on the server (PHP's session handler) and the client only holds an opaque session ID cookie. There is no endpoint in the application that allows one user to write to another user's row.

Restriction 3 – No Direct User Modification Routes:

The tetrisgame class exposes no methods for editing usernames, emails, or passwords after registration, and no methods for deleting user accounts. Even if a user somehow gained access to the PHP files, there is no code path that modifies any column other than score, and only for the currently authenticated user.

4.3 Edge Case Handling

The following edge cases were identified and handled in both the PHP back-end and JavaScript game engine:

Edge Case 1 – Boundary Collision Detection (hasCollision in script.js):

Every movement and rotation is pre-validated against three conditions before being applied: left/right wall bounds (x < 0 or x >= COLS), the floor (y >= ROWS), and existing placed blocks (board[y][x] is non-null). This single function is called before every move() and rotateTetromino() operation, making it impossible for a piece to visually leave the board or overlap another piece:

```
if (shape[y][x] && (currentPosition.x + x + offsetX < 0 ||
    currentPosition.x + x + offsetX >= COLS ||
    currentPosition.y + y + offsetY >= ROWS ||
    board[currentPosition.y + y + offsetY][currentPosition.x + x + offsetX])) {
    return true;
}
```

Edge Case 2 – Invalid Rotation Revert:

When the player rotates a piece, the new shape is applied first, and then hasCollision(0, 0) is called. If a collision is detected at the new orientation (e.g., against a wall or another block), the original shape is immediately restored. This prevents a rotation from "pushing" a piece through a wall or a placed block:

```
const originalShape = currentTetromino.shape;
currentTetromino.shape = newShape;
if (hasCollision(0, 0)) {
    currentTetromino.shape = originalShape; // revert
}
```

Edge Case 3 – Multiple Simultaneous Row Clears:

The removeRow() function scans from the bottom of the board upward. When a full row is removed via board.splice(y, 1) and a blank row is added at the top via board.unshift(), the loop counter y is incremented (y++) to re-check the same index, because the rows above have shifted down by one position. Without this correction, two adjacent full rows would result in only one being cleared:

```
for (let y = ROWS - 1; y >= 0; y--) {
    if (board[y].every(cell => cell)) {
        board.splice(y, 1);
        board.unshift(Array(COLS).fill(null));
        linesRemoved++;
        y++; // re-check same index after shift
    }
}
```

Edge Case 4 – Duplicate Score Submission Prevention:

A scoreSent boolean flag is initialized to false at the start of each game and set to true immediately after the first saveHighScore() call in the game loop's game-over block. This prevents the score from being submitted multiple times if requestAnimationFrame somehow fires again after game over, or if the user manually triggers a save before the page redirects:

```
if (gameOver) {
    if (!scoreSent) {
        saveHighScore();
        scoreSent = true;
    }
    return;
```

```
}
```

Edge Case 5 – NULL Score Handling:

When a user registers, their score column is NULL (no game played yet). Multiple null-coalescing operators (?? 0) are used throughout the PHP back-end to safely treat NULL as 0, preventing type errors and ensuring the leaderboard and dashboard display "0" for new players rather than a blank or PHP warning:

```
$currentScore = $stmt->fetchColumn() ?? 0;
$_SESSION['score'] = $user['score'] ?? 0;
return $stmt->fetchColumn() ?? 0;
```

Edge Case 6 – Unauthorized Page Access:

Both dashboard.php and leaderboard.php check for an authenticated session at the very top of the file, before any HTML is rendered. If the check fails, a redirect to login.php is issued immediately followed by exit() to ensure no further PHP code (which could expose data) is executed:

```
if (!isset($_SESSION['is_logged_in']) || $_SESSION['is_logged_in'] !== true) {
    $_SESSION['msg'] = "Please log in to access the dashboard.";
    header("Location: login.php");
    exit();
}
```

Edge Case 7 – Zero-Score Quit:

If a player quits immediately without scoring any points, the quitGame() function in script.js checks if score > 0 before calling saveHighScore(). This avoids sending a 0-score AJAX request that would trigger an unnecessary database query:

```
function quitGame() {
    if (score > 0) {
```

```
            saveHighScore().finally(() => {
                window.location.href = 'dashboard.php';
            });
            return;
        }
        window.location.href = 'dashboard.php';
    }
```

4.4 Garbage Line Attack System

Blitz Mode is not a pure parallel-score race. Clearing multiple lines on your own board sends garbage lines to the opponent's board, introducing a competitive offensive layer. The garbage table is the standard Tetris attack curve:

Lines Cleared	Garbage Lines Delivered
1 line clear	0 garbage sent
2 line clear (Double)	1 garbage sent
3 line clear (Triple)	2 garbage sent
4 line clear (Tetris)	4 garbage sent

Garbage rows are pushed onto the receiver's board from the bottom up. Each garbage row is a horizontal bar of neutral grey cells with one randomly chosen empty column, giving the receiver a chance to clear the rows if they align their next pieces around the hole. The "INCOMING!" banner flashes for 900 ms when garbage is added so the receiver is alerted to the change.

On the server side, garbage is tracked through two columns per player (p1_garbage, p2_garbage). The sync action increments the sender's garbage counter and the receiver reads the new total on its next sync, diffs it against lastOppGarbageSeen, and applies the delta. This transactional design makes the garbage delivery idempotent: even if a sync response is duplicated or arrives out of order, the receiver only ever applies each garbage chunk once.

4.5 Match-Ending Conditions

Blitz introduces several end-of-match conditions that did not exist in the single-player game. The endGame() function dispatches to the appropriate handler based on the reason, and the server's blitz_api->endGame() arbitrates which side is finalized when.

TIME_UP – The 2-minute timer hits zero. Whichever player has the higher score wins; equal scores produce a draw.

TOPPED_OUT – A new tetromino spawns into an already-occupied cell. Unlike single-player, the room is NOT immediately ended: the surviving opponent continues to play out the remaining time as a 1-player run, and the topped-out player enters spectator mode (myToppedBanner is shown, the rival canvas keeps updating). If both players top out, the match finalizes at that moment and the higher score wins.

OPPONENT_DISCONNECT – The server detects a missing opponent when the opp_updated timestamp is older than 12 seconds. The remaining player is declared the winner and the room is finalized with status = 'finished'. A best-effort sendBeacon on the beforeunload event also tells the server when a player closes the tab.

When the room status flips to 'finished' the blitz_leaderboard table is updated atomically with an INSERT ... ON CONFLICT (username) DO UPDATE upsert that increments wins, losses, and total_games, and only overwrites best_score if the new score is higher (using GREATEST()). This single-statement upsert prevents the double-update race that would otherwise be possible if both clients reported the end at roughly the same instant.

4.6 Rematch System

The result screen offers an in-place rematch flow so players do not have to navigate back to the lobby between games. The rematch is a fully arbitrated invite system with three terminal states (accepted, declined, expired) and a hard limit of two rematches per pairing to prevent infinite tab-camping. The relevant metadata is stored on the old room row itself:

Column	Type	Purpose
rematch_code	VARCHAR(8)	Pre-allocated room code for the rematch. Created when the first player clicks Rematch.
rematch_requested_by	VARCHAR(100)	Username of the player who initiated the rematch

		invite.
rematch_requested_at	TIMESTAMPTZ	Server timestamp of the invite. Used to compute the 15-second TTL window.
rematch_declined	INTEGER	Set to 1 when the invitee clicks Decline (or the invite auto-expires).
rematch_count	INTEGER	Counter that is incremented on every rematch and capped at 2 by REMATCH_LIMIT.

Flow: Player A clicks Rematch. The server records rematch_requested_by and rematch_requested_at on the old room, and inserts a new blitz_rooms row whose p1 is A. Player B then sees "A wants a rematch!" on their result screen with an Accept / Decline pair and a live 15-second countdown. Accept makes B p2 of the new room and redirects both players into blitz_room.php?mode=join&code=<new>. Decline (or timeout) sets rematch_declined = 1, clears the pointers, and deletes the orphan new room if nobody has joined it yet. A subsequent Rematch from either player goes through the full invite flow again until rematch_count reaches REMATCH_LIMIT = 2.

Several edge cases are handled by the server's rematch handlers: (1) if both players click Rematch within the TTL window, the second click is silently routed into rematch_accept so the players end up in the same room rather than two parallel rooms; (2) a re-request from the same player refreshes the TTL and clears any prior decline; (3) if the pre-allocated rematch room has already been cleaned up by the stale-room sweeper, the server forces the invite to look declined so the requester can issue a fresh one cleanly.

4.7 Ghost Piece (Landing Preview)

Both the single-player and multiplayer engines now render a translucent "ghost" outline at the row where the active tetromino would land if it dropped straight down. The drawGhost() function projects the active piece one row at a time until hasCollision() returns true, then paints the projected shape at 22 percent alpha with a colored outline. This dramatically reduces misdrops on the wider 10-column board and was added after early playtesting feedback.

4.8 Hard Drop (Slam)

Pressing Spacebar in Blitz Mode triggers a hard drop: the active piece is teleported to its ghost position and locked instantly. This is essential at competitive speeds and is the standard "Slam" control surfaced in the on-screen Controls panel of the game phase.

4.9 Touch Controls (Mobile)

Both single-player and Blitz Mode are playable on touchscreens. The Blitz game phase renders four on-screen buttons (Left, Turn, Drop, Right) below the boards on devices narrower than the medium breakpoint. In addition, swipe gestures on the canvas are recognized:

Short tap (< 18 px movement): rotate the active piece.

Horizontal swipe: move the piece left or right.

Downward swipe: hard-drop and lock the piece.

Upward swipe: rotate (alternative to tap).

4.10 CSRF Protection for Score Submissions

A CSRF (Cross-Site Request Forgery) token is now generated per session using bin2hex(random_bytes(16)) and stored in $_SESSION['csrf_token']. Each protected

page (game.php, blitz_room.php) emits the token in a <meta name="csrf-token"> tag or a JavaScript constant. The score-saving fetch in script.js reads the token from the meta tag and includes it in the request body. On the server, dashboard.php uses hash_equals() for a timing-safe comparison and rejects any submission whose token does not match with HTTP 403. Without this layer, a malicious cross-origin page could submit forged score updates against an authenticated user who happens to be logged in.

4.11 Database Resilience and Friendly Error Messages

The dbConnect/dbconnect.php class was hardened to make connection failures actionable rather than cryptic. Several pieces work together:

A connect_timeout=5 parameter is appended to the DSN so a paused or unreachable Supabase instance fails in 5 seconds instead of the default ~60 seconds, keeping the UI responsive when the project is asleep.

Before opening a connection the code checks extension_loaded('pdo_pgsql'). If the PostgreSQL driver is disabled in php.ini, the error message explicitly tells the developer which extensions to enable and to restart Apache.

Caught PDOException messages are mapped to friendlier text via friendlyError(): authentication failures, connection refused / timeouts, unknown host, and the very common "timeout expired" case (which on Supabase nearly always means the free-tier project has been auto-paused for inactivity).

blitz_api.php wraps every dispatch in a 2-attempt retry that recognizes transient PostgreSQL errors ("server closed the connection unexpectedly", "terminating connection due to administrator command", "SSL connection has been closed unexpectedly") and re-opens the pool before failing.

All blitz_api.php output is buffered with ob_start() so that stray PHP notices or warnings cannot corrupt the JSON response. The jsonOut() helper calls ob_clean() before echoing the final JSON.

4.12 Disconnect Handling and Beacon Cleanup

Three layers cooperate to detect and recover from disconnected opponents in Blitz Mode:

Heartbeat: every successful sync updates the caller's p1_updated or p2_updated timestamp. If the server sees that the opponent's timestamp is older than 12 seconds, it flags opp_disconnected = true in the next sync response.

Client-side reaction: on receiving opp_disconnected = true, the surviving client immediately calls endGame('OPPONENT_DISCONNECT') which reports reason=disconnected to the server. The server finalizes the room with the survivor as the winner.

Beacon on unload: the window.beforeunload handler issues a navigator.sendBeacon() POST to action=leave so the server can tear down or downgrade the room immediately rather than waiting for the 12-second heartbeat to lapse. Beacons are used because regular fetch requests are not guaranteed to complete during page navigation.

4.13 SPA-Style Page Architecture for Blitz Rooms

The blitz_room.php document is composed from seven phase partials (bootstrap.php, phase_error.php, phase_cooldown.php, phase_waiting.php, phase_ready.php, phase_countdown.php, phase_game.php, phase_result.php, boot_script.php). Each partial is a self-contained block of HTML, and the multiplayer.js controller toggles a single d-none class on each block to advance the visible phase. The result is a single-page-application feel — no page reloads between waiting, ready, countdown, game, and result — without introducing a JavaScript framework. The cache-buster query on the script include (multiplayer.js?v=<filemtime>) forces the browser to refetch the controller whenever the file changes.

5. IMPROVEMENT / REVISION

Several improvements and revisions were made from the initial implementation to the final submitted version. These revisions improved both the user experience and the technical robustness of the system.

Revision 1 – Addition of the "Next Piece" Preview Panel:

The first version of the game board had only the canvas and a score display. The "Next Piece" panel was added in a later iteration after playtesting revealed that players found it difficult to plan moves without knowing what piece was coming next. The panel dynamically renders the upcoming tetromino as a grid of colored div elements inside a dedicated nextPiece container.

Revision 2 – Speed Increase Interval Adjustment:

The initial speedIncreaseInterval was set to 10,000 ms (10 seconds). This was reduced to 5,000 ms (5 seconds) after testing showed that the original pace kept the game too easy for too long. The minimum interval cap of 500 ms was also added to prevent the fall speed from becoming impossibly fast.

Revision 3 – Switching from setInterval to requestAnimationFrame:

The original game loop used setInterval() to call moveDown() at a fixed rate. This was replaced with requestAnimationFrame() combined with a timestamp-based delta check. The benefit is that requestAnimationFrame syncs to the display refresh rate (60 fps), producing smoother rendering, and automatically pauses when the browser tab is hidden (preventing runaway game state updates in the background).

Revision 4 – Personal Ranking Added to Leaderboard:

The initial leaderboard page only showed the global top-10 table. A "Your Ranking" section was added below the table, showing the logged-in player's global rank, score, and username. This required adding the getUserRanking() method to the tetrisgame class, which uses a COUNT(*) + 1 query to compute the player's position efficiently without fetching all records.

Revision 5 – Score Session Caching:

Originally, the dashboard queried the database every time it loaded to display the player's personal best. This was optimized by caching the score in $_SESSION['score'] during login and updating the session value whenever UpdateScore() stores a new high score. The dashboard now calls $auth->getScore($_SESSION['username']) directly, which is a lightweight indexed lookup, and the session ensures the value shown is always current.

Revision 6 – Spectator Mode After Top-Out:

The original Blitz implementation ended the entire match the moment either player topped out, which felt unfair to a player who built a fast lead and then made one bad placement. The endGame() flow was rewritten so a single TOPPED_OUT keeps the room playing for the surviving opponent until the timer runs out. The topped-out player remains on the game phase as a spectator (the rival canvas keeps updating, the timer keeps ticking) until the room is finalized.

Revision 7 – Opponent Canvas Renders at 60 fps:

Early builds painted the opponent canvas only inside the sync response handler, which made the rival board look like it was stepping at 10 frames per second. The fix was to cache the latest snapshot in oppBoardCache and re-render it from the gameLoop's requestAnimationFrame tick, decoupling the visual frame rate from the network sync rate.

Revision 8 – Immediate Sync After Lock:

Because syncs only fired every 100 ms, the opponent could miss a freshly locked piece by up to ~100 ms. lockPiece() now invokes syncRoom() directly at the moment of the lock, so the opponent sees the new settled state with the minimum possible network latency.

Revision 9 – Idempotent Garbage Delivery:

An early bug let the receiver double-apply garbage if a sync response was retried by the browser. The protocol now sends a monotonically increasing p[n]_garbage counter and the receiver diffs against lastOppGarbageSeen, so duplicate or out-of-order sync responses can never enqueue a garbage chunk twice.

Revision 10 – Rematch Limit and Invite TTL:

Early Blitz allowed unlimited rematches, which led to players camping the result screen indefinitely. A REMATCH_LIMIT constant (default 2) and a

REMATCH_INVITE_TTL constant (default 15 seconds) were added on both the server and the client. The result screen shows a live countdown and a "Rematch limit reached" banner once the pairing cap is hit.

Revision 11 – Stale Room Cleanup:

Without housekeeping, blitz_rooms accumulated abandoned rows from browser closes, crashed sessions, and quick-match cancellations. A probabilistic cleaner now runs on roughly 10 percent of writes (random_int(1,10) === 1) and deletes any row older than 15 minutes. This piggybacks on existing write traffic instead of adding a cron job.

Revision 12 – Session Lock Release for Concurrency:

10 syncs per second per user could not work because PHP's default file-session handler holds an exclusive lock for the duration of every request. Adding session_write_close() immediately after the auth check at the top of blitz_api.php releases the lock as soon as $_SESSION is no longer needed, removing the throughput ceiling for concurrent fetches from the same browser.

Revision 13 – AbortController Sync Timeouts:

A hung fetch (DNS hiccup, paused Supabase) could leave the syncInFlight flag set to true and freeze the entire poll loop. Each blitzApi() call is now wrapped in an AbortController with a 4-second timeout, so a stalled request is forcibly cancelled and the loop recovers within one tick.

Revision 14 – CSRF Token on Score Submission:

Score submissions originally trusted any same-origin request. A per-session CSRF token (bin2hex(random_bytes(16))) is now minted in the session, emitted into the page, and verified server-side with hash_equals(). Submissions without a valid token are rejected with HTTP 403.

6. ERRORS ENCOUNTERED WHILE DEVELOPING

The following errors and bugs were encountered during development. Each issue, its root cause, and how it was resolved are documented below.

Error 1 – PDO Connection Failure to Supabase:

Symptom: "Connection failed: SQLSTATE[08006] [7] connection refused" on all pages.

Root Cause: The initial connection string used port 5432 (PostgreSQL's default direct port). Supabase's cloud infrastructure requires connections to go through the pgBouncer connection pooler on port 6543 for serverless and short-lived script connections. Using port 5432 in a PHP environment causes connection timeouts because each PHP script creates a new connection that the pooler is not configured to accept on that port.

Resolution: Updated dbconnect.php to use port 6543 and the full Supabase pooler hostname (aws-1-ap-southeast-2.pooler.supabase.com). All connections succeeded immediately after the change.

Error 2 – Score Submitted Multiple Times on Game Over:

Symptom: The browser console showed multiple "Score saved successfully" logs after a single game over event, and the network tab showed 3–5 identical POST requests to dashboard.php.

Root Cause: The initial game-over handling called saveHighScore() directly inside the gameLoop() function's check. Because requestAnimationFrame was still queued for the next frame, gameLoop() was called again before the async Fetch request completed, triggering another saveHighScore() call.

Resolution: Introduced the boolean flag scoreSent, initialized to false at game start. The game-over block now checks if (!scoreSent) before calling saveHighScore() and immediately sets scoreSent = true. The gameLoop() function returns early on game over without re-queueing requestAnimationFrame, and scoreSent is reset to false in performRestart().

Error 3 – Simultaneous Row Clears Only Removing One Row:

Symptom: When two or more rows were completed in a single piece placement, only one row was cleared and the others remained on the board, causing the game to end prematurely.

Root Cause: The original removeRow() loop decremented y after each iteration (standard for-loop behavior). When a row at index y was removed via splice(), all rows above shifted down by one index. The loop then moved to y-1, skipping the row that had just moved into position y.

Resolution: Added y++ immediately after the splice/unshift operations inside the if block. Because the outer loop decrements y at the end of each iteration, the net effect is y stays at the same value, re-checking the index where rows shifted into place. This correctly handles any number of simultaneous row clears.

Error 4 – Headers Already Sent Warning:

Symptom: PHP "Warning: Cannot modify header information - headers already sent" errors appeared when attempting to redirect after setting a session message.

Root Cause: Some PHP files had a blank line or whitespace before the opening <?php tag, or echo/print statements were executed before the header() call. Because PHP sends HTTP headers lazily (the first output flushes them), any output before header() causes the warning.

Resolution: Ensured all PHP files begin with <?php on the very first character (no leading whitespace or BOM), and that all header() redirect calls appear before any HTML output. The AJAX handler in dashboard.php was restructured so that the JSON response and exit() happen at the top of the file, before any HTML is ever reached.

Error 5 – Leaderboard Displaying Empty Rows:

Symptom: The leaderboard table rendered correctly for named rows but showed empty <td> cells without a surrounding <tr> for some entries.

Root Cause: The PHP loop in leaderboard.php was using echo to print individual <td> elements but was missing the opening <tr> tag before the first <td> in each iteration. The closing </tr> was present, so browsers auto-corrected the HTML inconsistently.

Resolution: The loop was corrected to echo a complete <tr>...</tr> block per iteration. htmlspecialchars() was also added to all echoed values to prevent XSS from usernames containing HTML special characters.

Error 6 – Stray PHP Notices Corrupting JSON Responses:

Symptom: the JavaScript client would occasionally fail with "Server returned invalid JSON" even though the action had clearly succeeded on the database side.

Root Cause: a PHP notice ("undefined index", "trying to access array offset on null") was being written to the response body before the json_encode() output, breaking the JSON parser.

Resolution: blitz_api.php now wraps the entire request in ob_start() at the top and the jsonOut() helper calls ob_clean() before emitting the JSON, discarding any stray notice or warning text. As a defense-in-depth measure, the client also pretty-prints the raw response text on parse failure so the underlying PHP notice is visible in the browser console.

Error 7 – 10 Hz Sync Loop Was Actually 1 Hz:

Symptom: even though setInterval(syncRoom, 100) was running every 100 ms, the network panel showed Blitz requests finishing roughly once per second.

Root Cause: PHP's default file-based session handler holds an exclusive flock() on the session file for the duration of every request. Because every fetch from the same browser shares the same session ID, the requests were being serialized at the session lock rather than running concurrently. Each request was waiting on the previous one's lock release.

Resolution: session_write_close() is called in blitz_api.php immediately after the authentication and CSRF checks. After that point the rest of the request never touches

$_SESSION, so releasing the lock early is safe and the sync loop runs at its intended 10 Hz.

Error 8 – Both Players Created Their Own Rematch Rooms:

Symptom: when both players clicked Rematch within a fraction of a second of each other, two separate rematch rooms were created and each player ended up alone in a different one.

Root Cause: rematch_request unconditionally allocated a new room. There was no server-side check for an existing pending invite by the opponent.

Resolution: rematch_request now reads the current invite state first, and if the opponent has already requested an active rematch (rematchInviteIsActive() returns true) it transparently funnels the request into rematch_accept instead, so both players end up in the same room. The client also checks data.is_host === false in the response and jumps straight into the joined room.

Error 9 – Single Top-Out Was Ending Both Sides:

Symptom: as soon as either player topped out, the entire match ended immediately, robbing the survivor of their remaining time and any final scoring run.

Root Cause: endGame() on the server was finalizing the room regardless of the opponent's alive status. The original logic assumed both players' games ended together.

Resolution: endGame() now finalizes only when reason !== 'topped_out' OR the opponent is already dead OR there is no real opponent. For a single TOPPED_OUT against a still-alive opponent, the room is left in status='playing'. The surviving player's eventual TIME_UP or TOPPED_OUT call finalizes the room, with winner computed based on who is still alive and the scores.

Error 10 – Garbage Lines Double-Applied After Retry:

Symptom: a flaky network would occasionally cause the receiver's board to fill up with twice as many garbage rows as the sender had actually generated.

Root Cause: the original protocol sent the delta ("+2 garbage") on every sync, so if the same response was retried by the browser the receiver would apply the chunk twice.

Resolution: the protocol now sends p1_garbage / p2_garbage as a monotonically increasing total. The receiver caches the last value in lastOppGarbageSeen and applies only the diff, making the garbage delivery idempotent across any retry or out-of-order arrival.

Error 11 – Supabase Auto-Pause Looked Like a Crash:

Symptom: after a long period of inactivity, every Blitz request failed with a 60-second hang followed by a generic PDOException about timeout. Players assumed the application was broken.

Root Cause: Supabase's free tier auto-pauses inactive projects. The PostgreSQL pooler accepts the TCP connection but no backend is available to complete the Postgres handshake, so the connect call waits the default ~60 seconds before reporting a timeout.

Resolution: connect_timeout=5 is now baked into the DSN so the failure surfaces in 5 seconds. The friendlyError() helper detects the "timeout expired" signature and produces an actionable message ("The Supabase project is likely paused — open the dashboard and click Restore project"). The Blitz client maps the same error string into a Retry-enabled error phase.

Error 12 – Headers Already Sent in blitz_api.php:

Symptom: PHP "Warning: Cannot modify header information" appeared in error_log whenever the API tried to set Content-Type after a transient warning.

Root Cause: the same root issue as Error 4 in the single-player code, but harder to spot because the warning was masked inside the JSON output buffer.

Resolution: ob_start() is now the very first instruction in blitz_api.php (before session_start() and the include), guaranteeing that any output produced inside the included files is captured and the header("Content-Type: application/json") call always succeeds.